Serverless AQI Forecasting System

An End-to-End Machine Learning Pipeline for Predicting Air Quality Index

Case Study: Hyderabad, Sindh, Pakistan

Location: Hyderabad, Sindh, Pakistan (25.396°N, 68.358°E)

Repository: https://github.com/AimanShykh/Pearls_AQI_Predictor

Live Dashboard: https://pearlsaqipredictor123.streamlit.app/

1. Problem Statement

Air pollution is a persistent and worsening public health concern in South Asia, and Hyderabad, Sindh, Pakistan is no exception. Elevated concentrations of particulate matter and gaseous pollutants are strongly associated with respiratory illness, cardiovascular disease, and reduced quality of life, particularly for children, the elderly, and individuals with pre-existing conditions.

Despite this risk, residents typically have access only to the current, real-time Air Quality Index (AQI) for their city, with no reliable, freely available short-term forecast. Without a forward-looking view of air quality, people cannot proactively plan outdoor activities, adjust exercise routines, or take preventive health measures (e.g., wearing a mask, using an air purifier, keeping windows closed) ahead of a pollution spike.

This project addresses that gap by building a system that not only reports the current AQI, but forecasts it 24, 48, and 72 hours into the future, using historical patterns in pollution and weather data.

2. Objectives

The project was designed around the following core objectives:

- Build a fully automated pipeline that continuously collects air quality and weather data for Hyderabad, Sindh with no manual intervention.
- Engineer meaningful, time-aware features (temporal patterns, lag values, rolling statistics) that capture how AQI evolves over time.
- Train and evaluate multiple machine learning models to forecast AQI 24, 48, and 72 hours ahead, and automatically select the best-performing model for each horizon.
- Deploy the entire system using only free, serverless infrastructure — no dedicated server, virtual machine, or paid hosting required.
- Present forecasts, historical trends, and hazardous-AQI alerts through an accessible, interactive web dashboard.
- Ensure the system re-trains itself on a recurring schedule, so forecasts stay current as new data accumulates, without any manual retraining step.

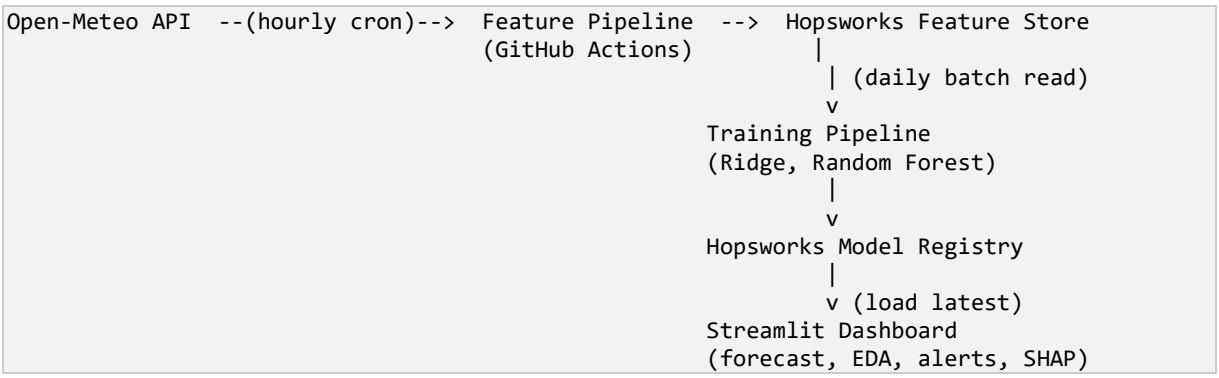
3. Proposed Solution / Approach

The solution is a serverless, event-driven machine learning pipeline composed of four cooperating stages, each automated on its own schedule:

- A Feature Pipeline that runs every hour, fetching the latest air quality and weather readings and converting them into model-ready features.
- A centralized Feature Store that persistently accumulates this hourly data over time, serving as the single source of truth for both training and prediction.
- A Training Pipeline that runs once a day, retraining candidate models on all data collected so far and registering the best model per forecast horizon.
- An Inference-driven Dashboard that loads the latest features and the latest trained models on demand, to display a live 3-day forecast.

4. System Architecture

The end-to-end data flow is as follows:



Each stage is a short-lived Python script, invoked on a schedule and then terminated — nothing runs continuously, which is the defining characteristic of a serverless design.

5. Data Sources

The choice of data source was not fixed from the start — it changed twice during development, each time for a concrete, documented reason.

4.1 Initial design — AQICN + OpenWeather

The first version of the pipeline used two providers: AQICN for ground-sensor AQI readings, and OpenWeather for weather and pollutant data. Both required API keys (free tier), and OpenWeather's historical weather endpoint specifically required a paid plan, while its historical pollution endpoint was free — an inconsistency that complicated the backfill design.

4.2 Simplification — switch to Open-Meteo

To reduce setup friction and the number of external dependencies, the project was rebuilt around a single provider: Open-Meteo. It requires no API key at all, already computes a standardized AQI value (us_aqi, 0-500 scale) rather than only raw pollutant concentrations, and — critically — offers full historical depth for both air quality and weather through the same interface, via explicit start_date/end_date parameters rather than a short, capped past_days window.

OTHER TESTINGS: A hybrid design — using AQICN for the live current reading while retaining Open-Meteo for weather and historical training data — was designed and code was drafted for it.

This hybrid approach was ultimately not deployed to production before the project deadline, after live testing revealed the model, trained entirely on Open-Meteo-scale values, produced an inflated 3-day forecast trend when fed a differently-scaled live AQICN reading. This was correctly diagnosed as a distribution mismatch between training-time and inference-time data sources, and the safer decision — reverting to Open-Meteo consistently across the whole pipeline — was made.

5. Development Log — Phase by Phase

Phase 1 — Feature Engineering Design

Input: raw hourly rows containing pm25, pm10, o3, no2, so2, co, temp, humidity, pressure, wind_speed, clouds.

Output: ~26 engineered columns across four groups — temporal (hour, day_of_week, month, cyclical sine/cosine encodings), lag features (AQI at 1h/6h/24h/48h prior), rolling/trend features (6h/24h rolling averages, hour-over-hour change rate), and three forward-shifted prediction targets (target_24h/48h/72h), built once in a single build_features() function shared by both training and inference to prevent training/serving skew.

A controlled experiment (comparing a model trained on raw columns only vs. the full engineered feature set) measured the real impact: RMSE dropped from approximately 18.4 to 7.1 on identical held-out data, confirming the engineered features — particularly the lag features — were doing the majority of the predictive work.

Phase 2 — Local Testing Strategy

Every pipeline script (backfill.py, training_pipeline.py, inference.py) was designed with a local-fallback mode: if HOPSWORKS_API_KEY is unset, each script reads/writes a local parquet file and local models/ folder instead of contacting Hopsworks. This allowed the core ML logic — feature engineering, model training, evaluation, inference — to be fully verified on a local machine before any cloud dependency was introduced, isolating logic bugs from infrastructure bugs.

Phase 3 — GitHub Repository & Actions Setup

Phase 4 — Historical Backfill (Initial, 90 days)

The first backfill run used Open-Meteo's simpler past_days parameter (capped at 92 days), producing an initial dataset of 2,184 hourly rows. This was sufficient to validate the end-to-end pipeline (fetch -> engineer -> store -> train -> predict) before committing to a larger, longer-running 2-year backfill.

Phase 5 — Extending to 2 Years of History

To give the model enough data to learn seasonal patterns (which a 90-day window cannot capture), the fetch logic was rewritten to use explicit start_date/end_date parameters instead of past_days. Weather required a separate endpoint for this depth — Open-Meteo's Archive API (ERA5 reanalysis, available back to 1940) — since the standard forecast-weather endpoint does not support multi-year lookback.

6. Challenges & Errors Encountered

This section documents every infrastructure-level failure encountered, in the order they occurred, including the exact diagnostic reasoning used to resolve each one.

6.1 Feature group creation — unsupported storage format

Feature group creation failed immediately after successful authentication.

Error observed:

```
hopsworks_common.client.exceptions.FeatureStoreException: Cannot use time_travel_format='DELTA':
delta library is not installed.
```

Root cause: Hopsworks defaults new feature groups to the Delta storage format, which requires an additional library not present in a minimal Python environment.

Resolution: The feature group creation call was updated to explicitly request time_travel_format="HUDI" (the standard, dependency-free format for this use case).

6.2 Missing Kafka client for data insertion

After fixing the storage format, inserting data into the newly created feature group failed at the write step.

Error observed:

```
ModuleNotFoundError: Confluent Kafka package not found. ... pip install "hopsworks[python]"
```

Root cause: Writing to a Hopsworks feature group internally requires a Kafka client library for streaming ingestion, which is not included in the base hopsworks package.

Resolution: requirements.txt was updated from hopsworks>=3.7.0 to hopsworks[python]>=3.7.0 to install the required extras.

6.3 Schema type mismatch on the 2-year backfill

The larger 2-year backfill failed where the smaller 90-day backfill had succeeded.

Error observed:

```
hopsworks_common.client.exceptions.FeatureStoreException: Features are not compatible with
Feature Group schema:
 - humidity (expected type: 'double', derived from input: 'bigint') has the wrong type.
 - clouds (expected type: 'double', derived from input: 'bigint') has the wrong type.
```

Root cause: The original 90-day backfill used the weather forecast endpoint, which returns humidity/clouds as decimals, fixing the feature group's schema to expect decimal (double) values. The 2-year backfill used the newly-added archive endpoint, which returns these same fields as whole integers, causing a type mismatch against the already-fixed schema.

Resolution: Explicit .astype(float) casts were added to the humidity and clouds columns in the archive-fetch function, forcing type consistency regardless of source endpoint.

6.4 SHAP explanation panel — intermittent library incompatibility

The dashboard's SHAP feature-importance panel intermittently failed to render.

Error observed:

```
'Client' object has no attribute '_project_id'
```

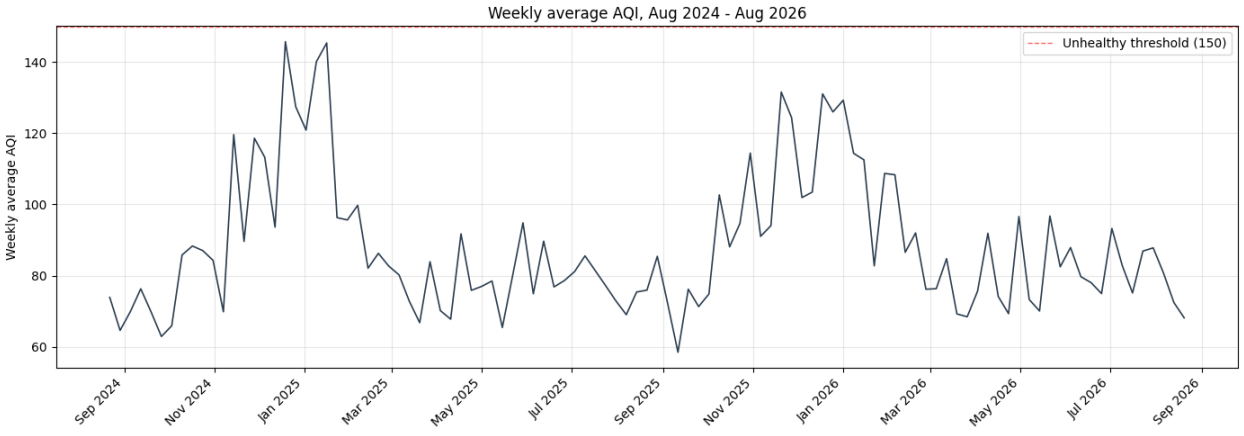
Root cause: This error originates inside the Hopsworks client library's internal state, likely from a stale connection reference retained by a model object loaded from the Model Registry, surfacing when SHAP's explainer inspects the model.

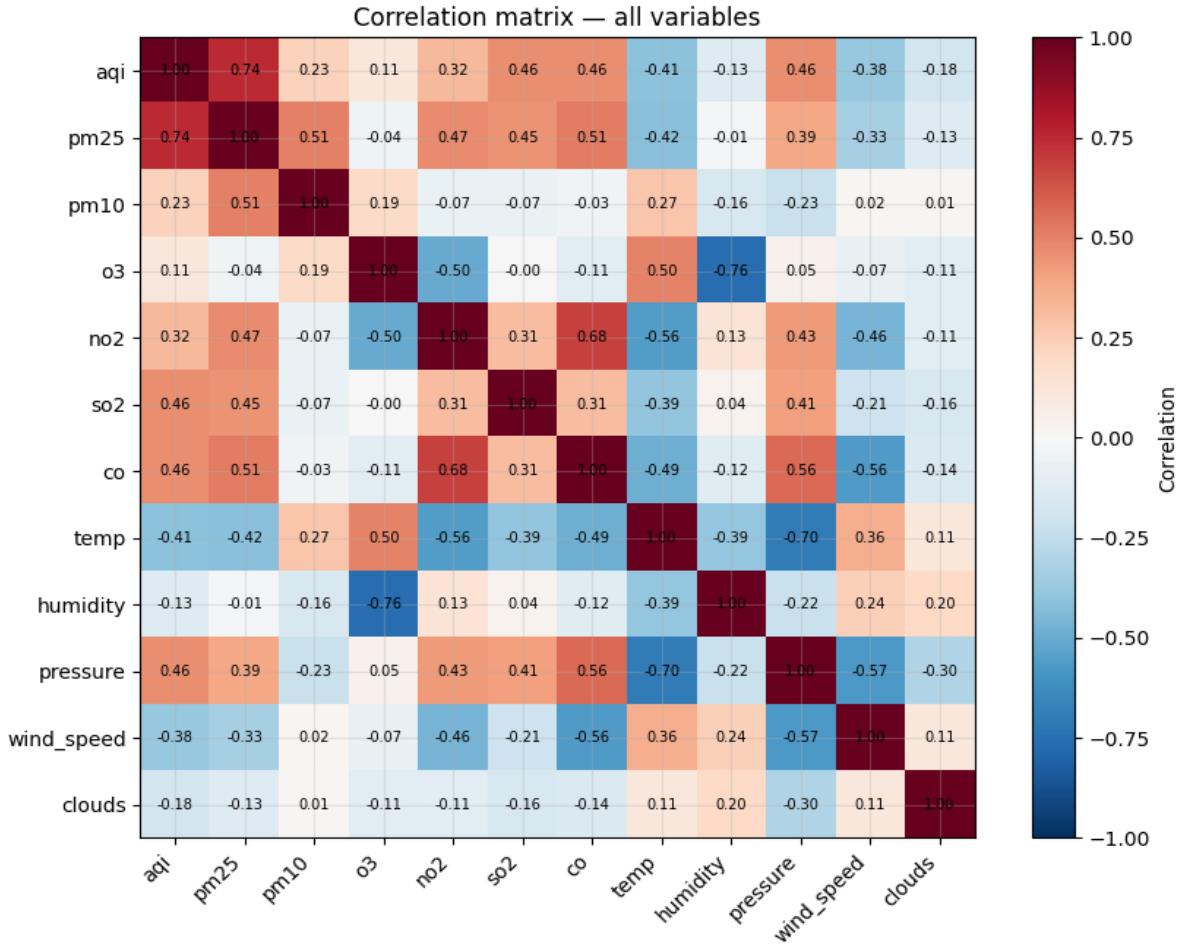
Resolution: Given proximity to the project deadline, this was deliberately handled defensively rather than root-caused further: the SHAP section is wrapped in a try/except block that displays a graceful "unavailable" notice on failure, ensuring the rest of the dashboard remains fully functional regardless of this specific panel's status.

7. Exploratory Data Analysis Findings

A dedicated EDA (see aqi_eda.ipynb in the repository) was performed on the full 2-year, 17,544-row dataset prior to model training, to validate data quality and understand the underlying patterns the model would need to learn.

Finding	Detail
Data completeness	Zero missing values across all columns and the full 2-year span
AQI category split	~76% Moderate, ~19% Unhealthy for Sensitive Groups, ~4% Unhealthy or worse
Seasonality	December/January averaged AQI ~116-120 (worst); September averaged ~70 (cleanest)
Daily rhythm	AQI rises from ~87 overnight to a peak of ~94 around 7-8pm local time
Strongest positive correlation	PM2.5 (r = 0.74), followed by CO and SO2
Strongest negative correlation	Temperature (r = -0.41) and wind speed (r = -0.38)



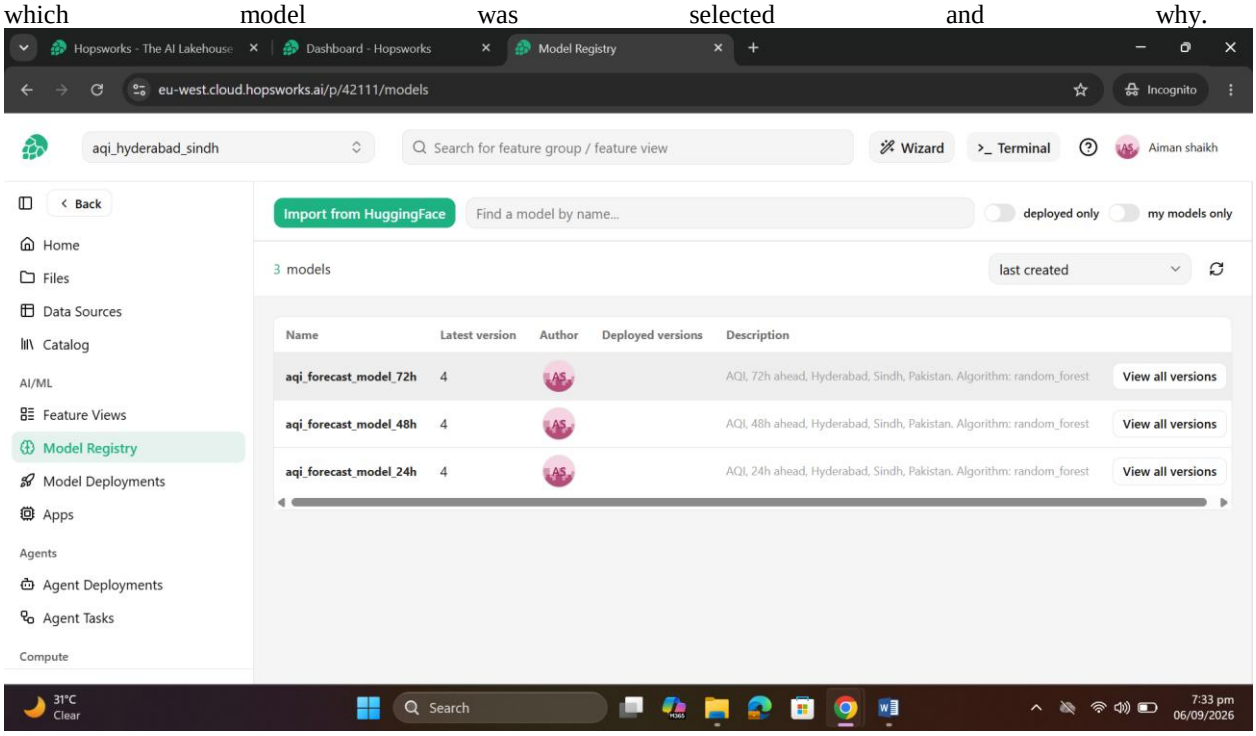


8. Model Training & Comparison

Model	Type	Role
Ridge Regression	Linear, regularized	Fast baseline
Random Forest	Tree ensemble	Captures non-linear relationships

Each candidate is scored with RMSE, MAE, and R^2 on the held-out test set; the lowest-RMSE model per horizon is automatically registered to the Hopsworks Model Registry as the production model for that horizon. The training script prints a direct comparison of all three candidates for every run, providing a transparent, auditable record of

which model was selected and why.

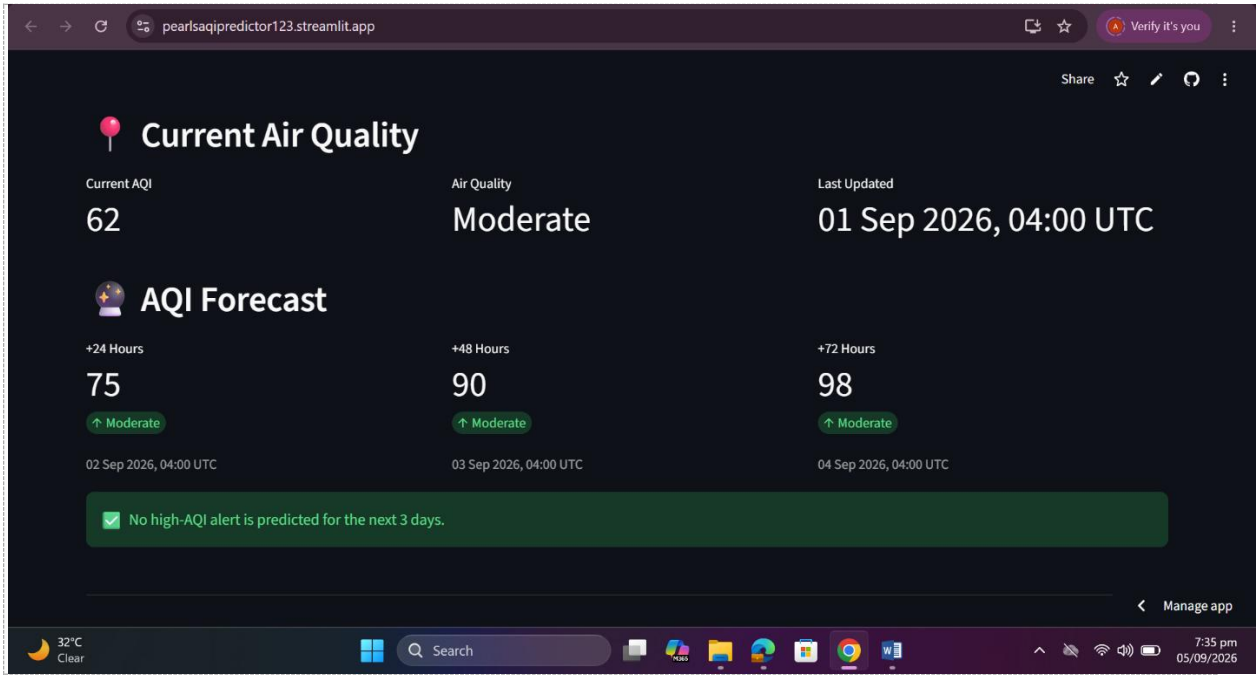


9. Dashboard — Features & Screenshots

The dashboard is built with Streamlit and deployed on Streamlit Community Cloud. It queries Hopsworks live on each page load for the latest features and latest registered models, meaning the displayed forecast is always current rather than pre-computed.

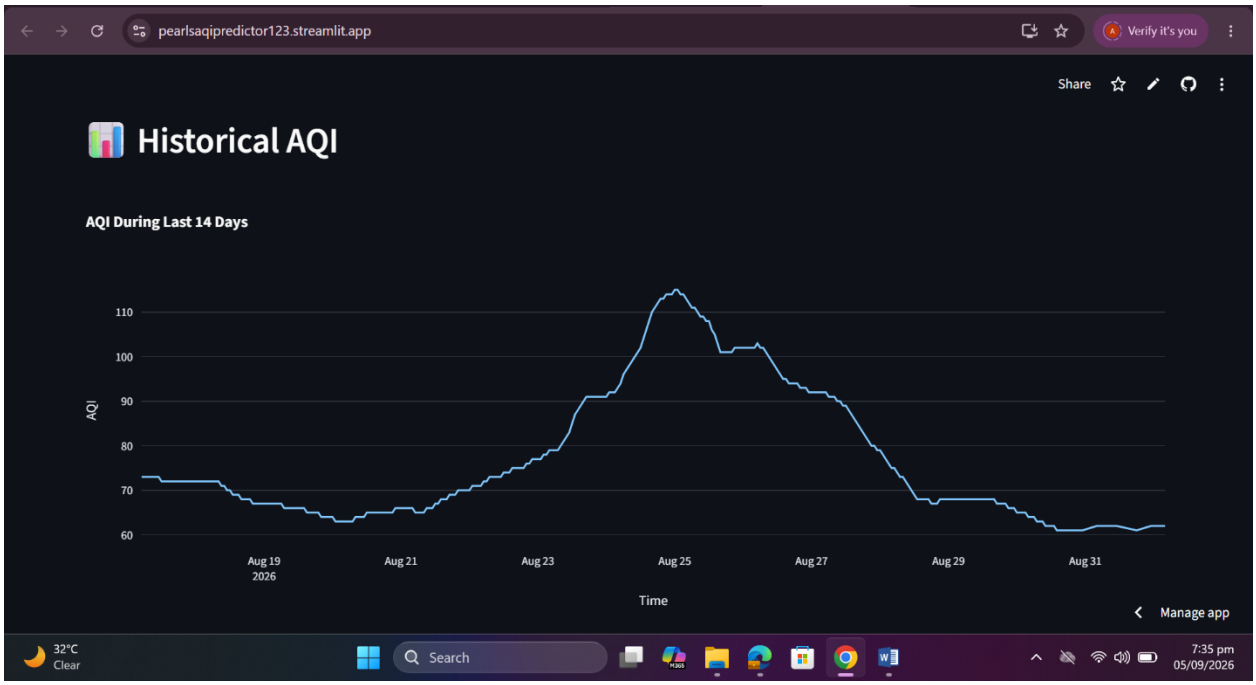
9.1 Main forecast view

- Current AQI and category, plus 24h/48h/72h forecast cards
- A trend line chart with AQI category bands overlaid
- An automatic red alert banner when any forecasted value reaches AQI >= 150



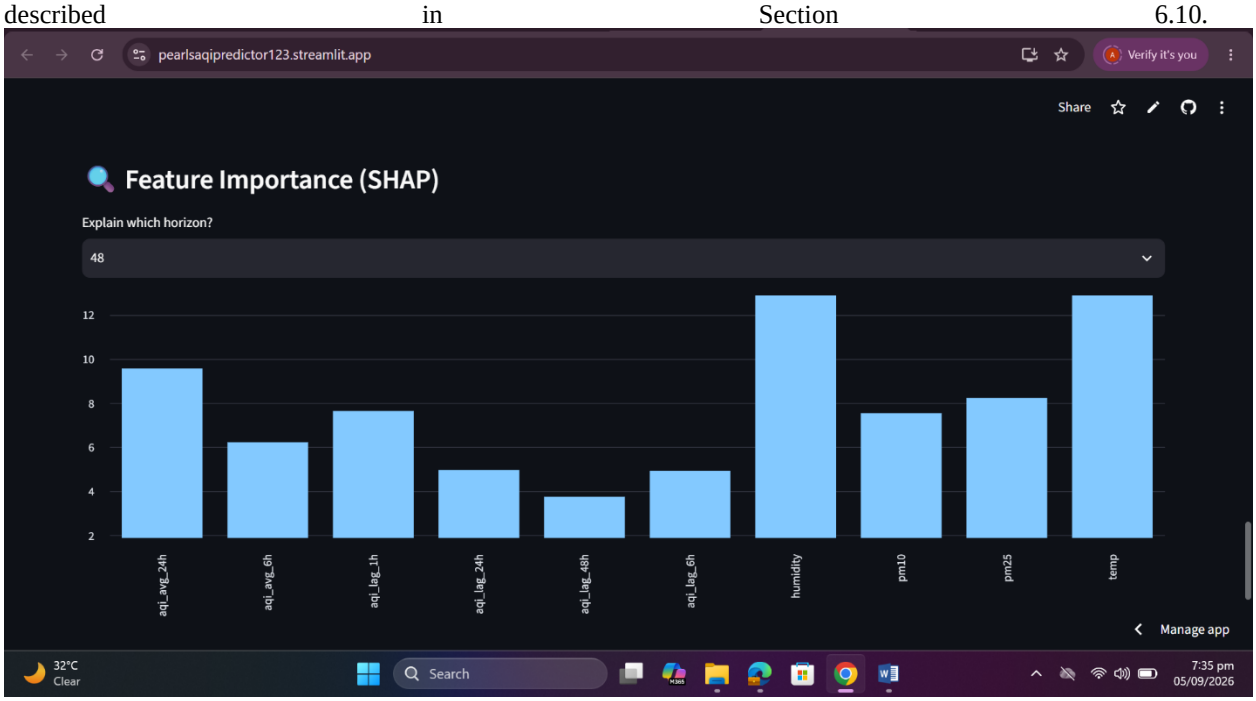
9.2 Historical trends

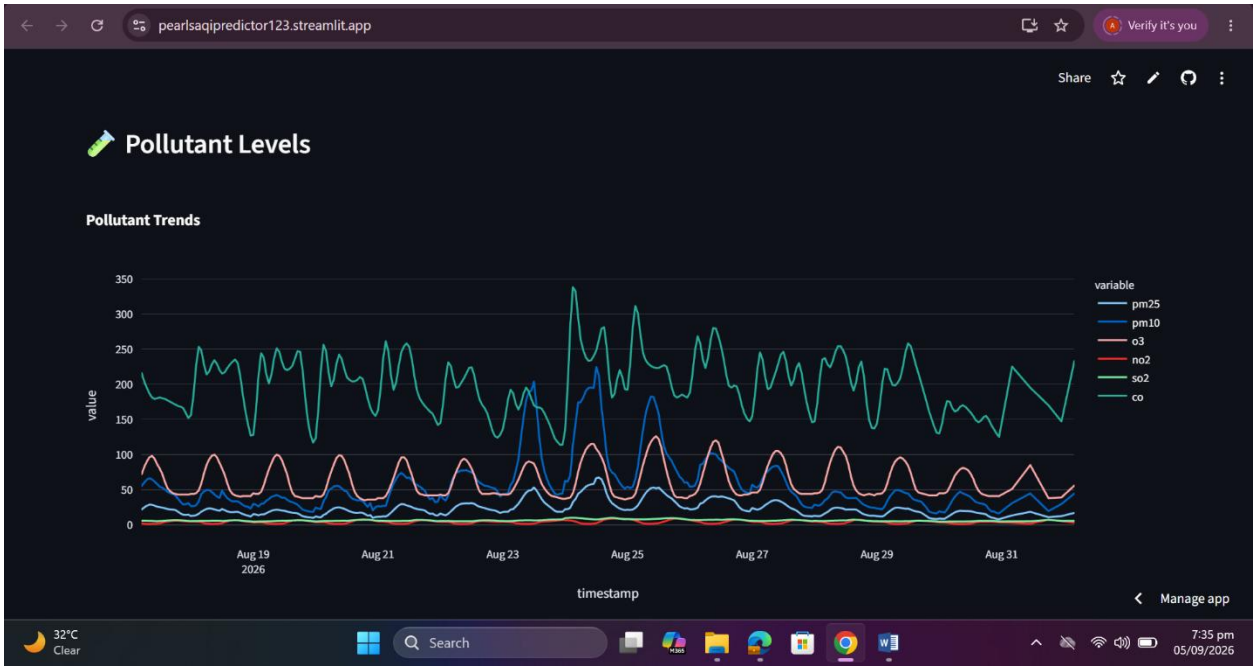
- 14-day AQI trend line
- AQI distribution by hour of day



9.3 SHAP feature importance

Shows which engineered features most influenced a given horizon's forecast, wrapped defensively per the fix described in Section 6.10.





10. Deployment Journey

Multiple hosting options were evaluated for the dashboard before settling on the final approach:

Option
Streamlit Community Cloud (CHOOSED)
Render (FastAPI)
Vercel / Gradio on Hugging Face